(19)国家知识产权局

(12)发明专利申请

(10)申请公布号 CN 116684385 A
(43)申请公布日 2023.09.01

(21)申请号 202310873481.6

(22)申请日 2023.07.17

(71)申请人 浙江大学
地址 310058 浙江省杭州市西湖区余杭塘路866号

(72)发明人 戴文清

(74)专利代理机构 杭州求是专利事务所有限公司 33200
专利代理师 邱启旺

(51)Int.Cl.
H04L 61/4511(2022.01)
H04L 49/90(2022.01)

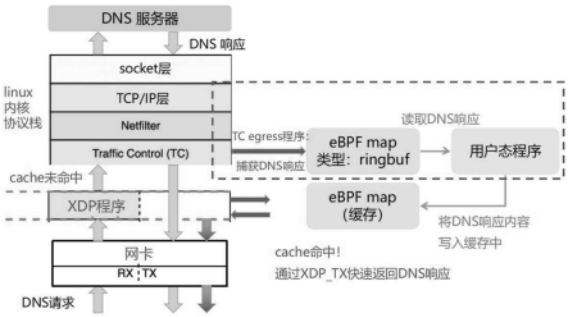
权利要求书2页 说明书11页 附图2页

(54)发明名称

一种基于eBPF在内核层面的DNS缓存方法

(57)摘要

本发明公开了一种基于eBPF在内核层面的DNS缓存方法，该方法在入口方向获取并解析DNS请求报文，根据解析后的DNS请求报文查找cache，构建DNS响应并快速返回；在出口方向获取并解析DNS服务器发出的响应报文并发送至控制面的ring_buffer，控制面再获取DNS响应报文携带的域名的哈希值和DNS响应内容，将DNS响应内容存储至cache中，并进行更新。本发明在网卡层解析并响应DNS请求，无需经过内核协议栈，实现了内核协议栈之前的DNS缓存，避免了内核协议栈的瓶颈；实现了离散分块存储，极大提升了内存利用率；显著降低了客户端的请求延迟，提升了DNS解析速度；显著降低了服务端的CPU消耗。



CN 116684385 A

1.一种基于eBPF在内核层面的DNS缓存方法,其特征在于,包括以下步骤:

(1)数据面:获取并解析DNS请求报文,根据解析后的DNS请求报文查找cache,以获取DNS响应报文,对DNS响应报文进行解析并将其发送至控制面的ring_buffer;

(2)控制面:通过ring_buffer获取DNS响应报文携带的域名的哈希值和DNS响应内容,将DNS响应内容存储至cache中,并对cache进行更新。

2.根据权利要求1所述的基于eBPF在内核层面的DNS缓存方法,其特征在于,所述步骤(1)包括以下子步骤:

(1.1)DNS服务器捕获网卡收到的来自于DNS客户端的所有数据包,对数据包进行过滤以获取DNS请求报文,对DNS请求报文进行解析,以获取DNS请求报文携带的域名的哈希值;

(1.2)根据DNS请求报文携带的域名的哈希值在cache的entry_map中查找与该哈希值匹配的元信息,根据元信息判断cache是否命中,若cache命中,则根据元信息以及响应内容修改DNS请求报文,将修改后的DNS请求报文作为DNS响应从DNS服务器的网卡发送到DNS客户端;否则,直接执行步骤(1.3);其中,所述元信息包括响应内容的过期时间、上次请求当前域名的时间、响应内容存放的位置以及响应内容的大小;

(1.3)DNS客户端捕获DNS服务器发出的响应,对该响应进行过滤以获取DNS响应报文,对DNS响应报文进行解析,以获取DNS响应报文携带的域名的哈希值和DNS响应内容;

(1.4)将DNS响应报文携带的域名的哈希值和DNS响应内容发送到控制面的ring_buffer。

3.根据权利要求2所述的基于eBPF在内核层面的DNS缓存方法,其特征在于,所述根据元信息判断cache是否命中具体包括:若根据域名的哈希值在cache的entry_map中查找到与该哈希值匹配的元信息,且元信息中响应内容的过期时间小于等于当前时间,则cache命中;若根据域名的哈希值在cache的entry_map中未查找到与该哈希值匹配的元信息或元信息中响应内容的过期时间大于当前时间,则cache未命中。

4.根据权利要求2所述的基于eBPF在内核层面的DNS缓存方法,其特征在于,所述根据元信息以及响应内容修改DNS请求报文具体包括:将cache中的响应内容添加到DNS请求报文后面;设置DNS请求报文的头字段;添加交换目的mac地址/ip/端口和源mac地址/ip/端口,在ip header和udp header中设置正确的长度,并重新计算ip层校验和。

5.根据权利要求1所述的基于eBPF在内核层面的DNS缓存方法,其特征在于,所述步骤(2)包括以下子步骤:

(2.1)控制面通过ring_buffer获取DNS响应报文携带的域名的哈希值和DNS响应内容;

(2.2)根据DNS请求报文携带的域名的哈希值在cache的entry_map中查找是否存在与该哈希值匹配的元信息,若entry_map中存在与该哈希值匹配的元信息且元信息中响应内容的过期时间大于当前时间,则删除entry_map和data_store中的元信息及DNS响应内容;否则,直接执行步骤(2.3);其中,所述元信息包括响应内容的过期时间、上次请求当前域名的时间、响应内容存放的位置以及响应内容的大小;

(2.3)根据元信息中响应内容的大小检测cache的data_store的剩余空间是否足够存储DNS响应内容,若data_store的剩余空间足够存储DNS响应内容,则直接执行步骤(2.4);否则,使用近似LRU方法驱逐旧的缓存条目,直至data_store的剩余空间足够存储DNS响应内容;

(2.4)将DNS响应内容分成固定大小的多个存储块,并将多个存储块存储至cache的data_store中;

(2.5)更新cache的entry_map:为entry_map添加一个新的条目,该条目包括键及其对应的键值,键为DNS请求报文携带的域名的哈希值,键值为<首个存储块索引,请求内容的大小,响应内容的大小,响应内容的过期时间,上次请求当前域名的时间>。

6.根据权利要求5所述的基于eBPF在内核层面的DNS缓存方法,其特征在于,所述使用近似LRU方法驱逐旧的缓存条目具体包括:首先从entry_map中随机读取k个键值对,其中键值对包括DNS请求报文携带的域名的哈希值及其对应的元信息;然后对比k个元信息中的上次请求当前域名的时间,将最早的上次请求当前域名的时间所对应的DNS请求报文携带的域名的哈希值从entry_map中删除;再根据元信息中的首个存储块索引和响应内容的大小删除该域名对应的DNS响应内容。

一种基于eBPF在内核层面的DNS缓存方法

技术领域

[0001] 本发明涉及DNS的缓存技术领域,尤其涉及一种基于eBPF在内核层面的DNS缓存方法。

背景技术

[0002] DNS是互联网中用于将域名解析为IP地址的系统,它是一个分布式数据库系统,用于存储和管理域名和IP地址之间的映射关系。作为互联网世界的重要基础设施,DNS的重要性不言而喻,它使得用户可以使用易于记忆的域名来访问互联网上的各种资源,DNS服务器还可以用于管理和维护网络资源,例如通过将域名映射到不同的IP地址来实现负载均衡和故障转移;在云原生场景下,coreDNS作为Kubernetes的核心组件之一,为Service的抽象提供了重要的支持。

[0003] 但在一些大规模场景下,DNS服务器可能会出现过载甚至崩溃的现象,这会影响到所有服务器域名的解析和访问,后果十分严重。因此,提高DNS服务器的性能和可靠性十分重要。

[0004] 为了解决这个问题,目前的主流方案是在DNS服务器之前添加一层cache。这里的cache是指DNS服务器端的缓存,用于存储最近访问的域名和IP地址之间的映射关系。当用户请求一个域名时,DNS服务端会首先检查它的缓存中是否已经存在该域名的映射关系,如果存在,则直接返回对应的IP地址;否则,DNS服务器会向其他DNS服务器发送查询请求,以获取该域名的IP地址,并将查询结果存储到缓存中,以便下次访问时使用。

[0005] DNS cache带来的优势包括:提高DNS查询的性能,减少客户端的请求延迟以及服务端的负载;DNS服务备份,即当DNS服务器异常时,DNS cache可以在缓存期内继续提供服务,缩短故障时间,提高可用性;DDoS缓解,即通过缓存DNS响应,可一定程度上缓解大量DNS DDoS流量的冲击,保护DNS服务器;在自建DNS的场景下,可以节省带宽和成本。

[0006] 然而,上述方案的DNS cache是通过用户态程序实现的,这意味着从DNS请求到来到DNS响应发出的整个过程都需要经过2次完整的内核协议栈。而内核协议栈是相当复杂的,有数据统计,在linux系统中,以memcached服务为例,内核协议栈处理UDP数据包所耗费的CPU百分比接近50%。也就是说,即便有了DNS cache,且在cache命中的情况,服务器仍需要耗费接近一半的CPU处理DNS数据包,在高并发的情况下,内核协议栈仍然可能成为服务器的瓶颈。

发明内容

[0007] 本发明的目的在于针对现有技术的不足,提供一种基于eBPF在内核层面的DNS缓存方法。

[0008] 本发明的目的是通过以下技术方案来实现的:一种基于eBPF在内核层面的DNS缓存方法,包括以下步骤:

[0009] (1)数据面:获取并解析DNS请求报文,根据解析后的DNS请求报文查找cache,以获

取DNS响应报文，对DNS响应报文进行解析并将其发送至控制面的ring_buffer；

[0010] (2)控制面：通过ring_buffer获取DNS响应报文携带的域名的哈希值和DNS响应内容，将DNS响应内容存储至cache中，并对cache进行更新。

[0011] 进一步地，所述步骤(1)包括以下子步骤：

[0012] (1.1)DNS服务器捕获网卡收到的来自于DNS客户端的所有数据包，对数据包进行过滤以获取DNS请求报文，对DNS请求报文进行解析，以获取DNS请求报文携带的域名的哈希值；

[0013] (1.2)根据DNS请求报文携带的域名的哈希值在cache的entry_map中查找与该哈希值匹配的元信息，根据元信息判断cache是否命中，若cache命中，则根据元信息以及响应内容修改DNS请求报文，将修改后的DNS请求报文作为DNS响应从DNS服务器的网卡发送到DNS客户端；否则，直接执行步骤(1.3)；其中，所述元信息包括响应内容的过期时间、上次请求当前域名的时间、响应内容存放的位置以及响应内容的大小；

[0014] (1.3)DNS客户端捕获DNS服务器发出的响应，对该响应进行过滤以获取DNS响应报文，对DNS响应报文进行解析，以获取DNS响应报文携带的域名的哈希值和DNS响应内容；

[0015] (1.4)将DNS响应报文携带的域名的哈希值和DNS响应内容发送到控制面的ring_buffer。

[0016] 进一步地，所述根据元信息判断cache是否命中具体包括：若根据域名的哈希值在cache的entry_map中查找到与该哈希值匹配的元信息，且元信息中响应内容的过期时间小于等于当前时间，则cache命中；若根据域名的哈希值在cache的entry_map中未查找到与该哈希值匹配的元信息或元信息中响应内容的过期时间大于当前时间，则cache未命中。

[0017] 进一步地，所述根据元信息以及响应内容修改DNS请求报文具体包括：将cache中的响应内容添加到DNS请求报文后面；设置DNS请求报文的头字段；添加交换目的mac地址/ip/端口和源mac地址/ip/端口，在ip header和udp header中设置正确的长度，并重新计算ip层校验和。

[0018] 进一步地，所述步骤(2)包括以下子步骤：

[0019] (2.1)控制面通过ring_buffer获取DNS响应报文携带的域名的哈希值和DNS响应内容；

[0020] (2.2)根据DNS请求报文携带的域名的哈希值在cache的entry_map中查找是否存在与该哈希值匹配的元信息，若entry_map中存在与该哈希值匹配的元信息且元信息中响应内容的过期时间大于当前时间，则删除entry_map和data_store中的元信息及DNS响应内容；否则，直接执行步骤(2.3)；其中，所述元信息包括响应内容的过期时间、上次请求当前域名的时间、响应内容存放的位置以及响应内容的大小；

[0021] (2.3)根据元信息中响应内容的大小检测cache的data_store的剩余空间是否足够存储DNS响应内容，若data_store的剩余空间足够存储DNS响应内容，则直接执行步骤(2.4)；否则，使用近似LRU方法驱逐旧的缓存条目，直至data_store的剩余空间足够存储DNS响应内容；

[0022] (2.4)将DNS响应内容分成固定大小的多个存储块，并将多个存储块存储至cache的data_store中；

[0023] (2.5)更新cache的entry_map：为entry_map添加一个新的条目，该条目包括键及

其对应的键值，键为DNS请求报文携带的域名的哈希值，键值为<首个存储块索引，请求内容的大小，响应内容的大小，响应内容的过期时间，上次请求当前域名的时间>。

[0024] 进一步地，所述使用近似LRU方法驱逐旧的缓存条目具体包括：首先从entry_map中随机读取k个键值对，其中键值对包括DNS请求报文携带的域名的哈希值及其对应的元信息；然后对比k个元信息中的上次请求当前域名的时间，将最早的上次请求当前域名的时间所对应的DNS请求报文携带的域名的哈希值从entry_map中删除；再根据元信息中的首个存储块索引和响应内容的大小删除该域名对应的DNS响应内容。

[0025] 本发明的有益效果是，本发明在网卡层解析并响应DNS请求，无需经过内核协议栈，实现了内核协议栈之前的DNS缓存，在网卡驱动层直接响应DNS请求，避免内核协议栈的瓶颈；本发明异步地将响应存储至cache中，避免内核态eBPF程序的诸多限制，可支持多种cache驱逐算法；本发明实现了离散分块存储，极大地提升了内存利用率；本发明显著降低了客户端的请求延迟，提升了DNS解析速度；本发明显著降低了服务端的CPU消耗，提高服务端能承受的最大QPS，提升服务健壮性和可靠性；本发明无需修改DNS服务器和内核，对上层完全透明；本发明可动态加载和卸载。

附图说明

[0026] 图1为本发明的基于eBPF在内核层面的DNS缓存方法的流程图；

[0027] 图2为本发明中离散固定大小分块存储算法的工作流程。

具体实施方式

[0028] 这里将详细地对示例性实施例进行说明，其示例表示在附图中。下面的描述涉及附图时，除非另有表示，不同附图中的相同数字表示相同或相似的要素。以下示例性实施例中所描述的实施方式并不代表与本申请相一致的所有实施方式。相反，它们仅是与如所附权利要求书中所详述的、本申请的一些方面相一致的装置和方法的例子。应当理解的是，以上的一般描述和后文的细节描述仅是示例性和解释性的，并不能限制本申请。

[0029] 在本申请使用的术语是仅仅出于描述特定实施例的目的，而非旨在限制本申请。在本申请和所附权利要求书中所使用的单数形式的“一种”、“所述”和“该”也旨在包括多数形式，除非上下文清楚地表示其他含义。还应当理解，本文中使用的术语“和/或”是指并包含一个或多个相关联的列出项目的任何或所有可能组合。

[0030] 应当理解，尽管在本申请可能采用术语第一、第二、第三等来描述各种信息，但这些信息不应限于这些术语。这些术语仅用来将同一类型的信息彼此区分开。例如，在不脱离本申请范围的情况下，第一信息也可以被称为第二信息，类似地，第二信息也可以被称为第一信息。取决于语境，如在此所使用的词语“如果”可以被解释成为“在……时”或“当……时”或“响应于确定”。而且，术语“包括”、“包含”或者其任何其他变体意在涵盖非排他性的包含，从而使得包括一系列要素的过程、方法不仅包括那些要素，而且还包括没有明确列出的其他要素，或者是还包括为这种过程、方法所固有的要素。在没有更多限制的情况下，由语句“包括一个……”限定的要素，并不排除在包括所述要素的过程、方法、物品或者设备中还存在另外的相同要素。

[0031] 本发明的基于eBPF在内核层面的DNS缓存方法能够在数据包进入到内核最早期的

阶段(若网卡支持,甚至可以完全offload至网卡处理)时捕获到DNS请求,若cache命中,则直接从网卡发出DNS响应,无需上交至内核协议栈和上层DNS服务器处理。整个过程完全是在内核协议栈之前完成的,最大程度地优化缓存效果,降低服务端负载,节省资源。同时,本发明可以在eBPF map中实现固定块大小链式离散存储,有助于提高eBPF map作为cache的内存利用率,可以避免浪费内核空间的内存,在内存固定的情况下存储更多的cache条目,提高缓存命中率。

[0032] 应当理解的是,eBPF(Extended Berkeley Packet Filter)是一种新兴的linux内核技术,它允许用户在不需要重新编译内核的情况下,在内核中注入自定义的代码,从而实现对内核的扩展和增强。eBPF最初是为网络数据包过滤和监控而设计的,但现在已经广泛应用于网络虚拟化、系统跟踪、安全监控、性能分析等领域。

[0033] 需要说明的是,eBPF程序有很多种类型,本发明所使用到的程序类型有2种,分别是XDP(eXpress Data Path)程序和TC(Traffic Controll)程序。XDP程序可以加载到物理或虚拟网卡上,在数据包到达网络接口的最早阶段进行处理,并且可以在内核中直接处理数据包,从而避免了用户态和内核态之间的切换,大大提高了数据包处理的性能和效率。TC程序可以attach到内核协议栈的ingress或egress路径上,捕获进出协议栈的数据包。

[0034] 本发明所使用的程序类型除了上述eBPF程序外,还需使用到若干eBPF map,eBPF map是Linux内核中的一种数据结构,用于在用户态程序和内核态eBPF程序中读取和存储数据。为了实现DNS缓存,需要使用eBPF map作为cache的实际后端存储。

[0035] 本发明将TC程序加载到内核协议栈的egress方向,捕获上层DNS服务器发出的响应发送给用户态程序,由用户态程序将响应内容更新到cache中;将XDP程序加载到物理网卡上,由XDP程序在内核协议栈之前完成cache查找和返回,逐步在内核层面上实现DNS缓存,有利于优化缓存效果,降低服务端负载,节省资源。

[0036] 参见图1,本发明所述的基于eBPF在内核层面的DNS缓存方法,具体包括以下步骤:

[0037] (1)数据面(内核态):获取并解析DNS请求报文,根据解析后的DNS请求报文查找cache,以获取DNS响应报文,对DNS响应报文进行解析并将其发送至控制面的ring_buffer。

[0038] 需要说明的是,数据面的执行过程包括两个XDP程序和两个TC程序。

[0039] (1.1)DNS服务器捕获网卡收到的来自于DNS客户端的所有数据包,对数据包进行过滤以获取DNS请求报文,对DNS请求报文进行解析,以获取DNS请求报文携带的域名的哈希值。

[0040] 具体地,DNS服务器捕获网卡收到的来自于DNS客户端的所有数据包之后,可以预先对数据包的目的端口号进行配置,例如将目的端口号配置为53,那么如果数据包的端口号为53,则该数据包对应的数据内容即为DNS请求报文。然后根据DNS的标准格式对DNS请求报文进行解析,即按照DNS的标准格式读取DNS请求报文的域名,使用32位的FNV-1a哈希函数计算哈希值,遍历DNS请求报文的每个字节,边读取边计算哈希值,每读取一个DNS请求报文的字节,都会增量地计算一次哈希值,累计计算哈希值,在解析DNS请求报文的同时,顺带计算出哈希值,最后得到了DNS请求报文携带的域名的哈希值。综上所述,将该过程定义为xdp_rx_filter。

[0041] (1.2)根据DNS请求报文携带的域名的哈希值在cache的entry_map中查找与该哈希值匹配的元信息,根据元信息判断cache是否命中,若cache命中,则根据元信息以及响应

内容修改DNS请求报文，将修改后的DNS请求报文作为DNS响应从DNS服务器的网卡发送到DNS客户端；否则，直接执行步骤(1.3)。其中，元信息包括响应内容的过期时间、上次请求当前域名的时间、响应内容存放的位置以及响应内容的大小。

[0042] 本实施例中，根据元信息判断cache是否命中具体包括：若根据域名的哈希值在cache的entry_map中查找到与该哈希值匹配的元信息，且元信息中响应内容的过期时间小于等于当前时间，则cache命中；若根据域名的哈希值在cache的entry_map中未查找到与该哈希值匹配的元信息或元信息中响应内容的过期时间大于当前时间，则cache未命中。

[0043] 本实施例中，根据元信息以及响应内容修改DNS请求报文具体包括：将cache中的响应内容添加到DNS请求报文后面，即将响应的具体内容例如域名的ip地址等添加到DNS请求报文的后面，表示这是对应的应答；设置DNS请求报文的头字段，用于表明该请求报文是一个DNS响应，有几个应答等等；添加交换目的mac地址/ip/端口和源mac地址/ip/端口，在ip header和udp header中设置正确的长度，并重新计算ip层校验和，如此才能发回给DNS客户端。

[0044] 具体地，entry map的key是域名的哈希值，因此根据步骤(1.1)中获取的域名的哈希值在entry_map中查找与该哈希值匹配的元信息，value只是元信息，并不包含实际的响应内容，data_store用于存储实际的响应内容，若能够查找到与该哈希值匹配的元信息，且元信息中响应内容的过期时间小于等于当前时间，则表示cache命中；否则，说明cache未命中。当cache命中的时候，根据元信息以及响应内容修改DNS请求报文，将修改后的DNS请求报文作为DNS响应从DNS服务器的网上发送到DNS客户端，如将XDP程序的返回值设置为XDP_TX，可以直接将网卡收到的报文反向从该网卡发出去，发出去的就是在XDP程序中修改后的DNS请求报文，即DNS响应，从DNS服务器的网卡发送到DNS客户端。当cache未命中的时候，则不做任何处理，交由上层DNS服务器处理即可。综上所述，将该过程定义为xdp_rx_response。

[0045] 应当理解的是，上层DNS服务器指的是DNS服务端的DNS服务器。

[0046] (1.3)DNS客户端捕获DNS服务器发出的响应，对该响应进行过滤以获取DNS响应报文，对DNS响应报文进行解析，以获取DNS响应报文携带的域名的哈希值和DNS响应内容。

[0047] 应当理解的是，对DNS响应报文进行解析后得到的不止包含域名的哈希值，还包括DNS响应内容，其中DNS响应内容包括但不限于域名对应的ip地址、DNS响应的过期时间、DNS响应的大小等等。

[0048] 具体地，DNS客户端捕获DNS服务器发出的响应，可以预先对报文的源端口号进行配置，如将其配置为53，那么如果报文的源端口号为53，则该报文即为对响应过滤后得到的DNS响应报文。然后根据DNS的标准格式对DNS响应报文进行解析，即按照DNS的标准格式读取DNS响应报文的域名，使用32位的FNV-1a哈希函数计算哈希值，遍历DNS响应报文的每个字节，边读取边计算哈希值，每读取一个DNS响应报文的字节，都会增量地计算一次哈希值，累计计算哈希值，在解析DNS响应报文的同时，顺带计算出哈希值，最后得到了DNS响应报文携带的域名的哈希值和DNS响应内容，如域名对应的ip地址、DNS响应的过期时间、DNS响应的大小等等。综上所述，将该过程定义为tc_tx_filter。应当理解的是，TC程序可以捕获到网卡发出的所有报文，通过上述逻辑即可过滤出DNS响应。

[0049] (1.4)将DNS响应报文携带的域名的哈希值和DNS响应内容发送到控制面的ring_

buffer。

[0050] 应当理解的是，步骤(1.4)是将步骤(1.3)中对DNS响应报文进行解析出的所有内容发送至控制面的ring_buffer。

[0051] 具体地，将DNS响应报文携带的域名的哈希值和DNS响应内容发送到控制面的ring_buffer，让控制面更新cache，缓存域名的哈希值和ip地址以及DNS响应内容。综上所述，将该过程定义为tc_tx_update_cache。

[0052] (2)控制面(用户态)：通过ring_buffer获取DNS响应报文携带的域名的哈希值和DNS响应内容，将DNS响应内容存储至cache中，并对cache进行更新。

[0053] (2.1)控制面通过ring_buffer获取DNS响应报文携带的域名的哈希值和DNS响应内容。

[0054] 具体地，控制面监听ring_buffer，在有新的DNS响应时读取ring_buffer中的DNS响应报文携带的域名的哈希值和DNS响应内容，之后再根据后续步骤对这些内容进行处理。

[0055] (2.2)根据DNS请求报文携带的域名的哈希值在cache的entry_map中查找是否存在与该哈希值匹配的元信息，若entry_map中存在与该哈希值匹配的元信息且元信息中响应内容的过期时间大于当前时间，则删除entry_map和data_store中的元信息及DNS响应内容，可以清除不再有效的缓存内容(DNS响应内容)，清空占用内存，便于后续缓存DNS响应内容；否则，直接执行步骤(2.3)。其中，元信息包括响应内容的过期时间、上次请求当前域名的时间、响应内容存放的位置以及响应内容的大小。

[0056] 应当理解的是，data_store本质上是一个数组，每个元素类型可以表示为：

[0057]
```
struct block {
    __u32 next;              // 下一块的索引
    __u8 raw[BLOCK_SIZE];    // 实际的数据（DNS 响应的一部分）
}
```

[0058] 应当理解的是，控制面会维护一个头指针head，初始时head为0。示例性地，当删除某个index块的内容时，可以使用：

[0059] data_store[index]＝{

[0060] next:head

[0061] }

[0062] head＝index

[0063] (2.3)根据元信息中响应内容的大小检测cache的data_store的剩余空间是否足够存储DNS响应内容，若data_store的剩余空间足够存储DNS响应内容，则直接执行步骤(2.4)；否则，使用近似LRU方法驱逐旧的缓存条目，直至data store的剩余空间足够存储DNS响应内容。

[0064] 本实施例中，使用近似LRU方法驱逐旧的缓存条目具体包括：首先从entry_map中随机读取k个键值对，其中键值对包括DNS请求报文携带的域名的哈希值及其对应的元信息；然后对比k个元信息中的上次请求当前域名的时间，将最早的上次请求当前域名的时间所对应的DNS请求报文携带的域名的哈希值从entry_map中删除；再根据元信息中的首个存储块索引和响应内容的大小删除该域名对应的DNS响应内容。

[0065] 具体地，从entry_map中随机读取的键值对的个数可以配置，例如将k配置为5，表示从entry_map中随机读取5个键值对；其中，键值对包括键及其对应的键值，键为DNS请求报文携带的域名的哈希值，键值为元信息<首个存储块索引，请求内容的大小，响应内容的大小，响应内容的过期时间，上次请求当前域名的时间>。对比k个元信息中的上次请求当前域名的时间，选出其中次请求当前域名的时间的一个键，从entry_map中将该键删除，即将DNS请求报文携带的域名的哈希值从entry_map中删除。根据元信息中的首个存储块索引和响应内容的大小删除该域名对应的DNS响应内容，即从首个存储块索引开始，顺着next指针一直向后遍历，将该域名对应的DNS响应内容从data_store中逐个将存储块删除。之后，循环重复使用近似LRU方法驱逐旧的缓存条目，直至data_store的剩余空间足够存储DNS响应内容。这种方法能够达到和近似LRU驱逐的效果，同时避免了严格LRU算法的额外计算和维护成本。

[0066] (2.4)将DNS响应内容分成固定大小的多个存储块，并将多个存储块存储至cache的data_store中。

[0067] 需要说明的是，可以预设一个固定大小，然后将DNS响应内容分成固定大小的多个存储块，再将多个存储块存储至cache的data_store中。

[0068] 应当理解的是，cache的底层存储是由eBPF map实现的，eBPF map也有很多种类，为了实现键值存储，一个直观的想法是使用BPF_MAP_TYPE_HASH类型的map，即哈希表，其键(key)为域名查询的哈希值，值(value)为元信息，可以表示实际的响应内容，但是并不存储实际的响应内容。其中，eBPF map的键和值的大小都是固定的，为了存储上层DNS服务器发出的响应，必须将值的大小设置为响应大小的最大值。但绝大多数的响应大小是远小于最大值的，这会导致非常严重的内存浪费，即只有一小部分值的内存实际用于存储，因此这种方案的缓存内存利用率非常不理想。因此，可以将DNS响应分成固定大小的存储块，将这些存储块离散地存储在一个BPF_MAP_TYPE_ARRAY类型的eBPF map中，map的每一个元素对应一个块，如图2所示。这种方式极大减少了内存碎片，显著提高了内存利用率。由于内核态eBPF程序的诸多限制，更新缓存的过程由用户态的控制面程序执行。

[0069] 示例性地，当存储一个块时，可以使用下列内容：

[0070] block＝data_store[head]

[0071] nxt＝block未初始化?head+1:block.next

[0072] data_store[head]＝{

[0073] next:nxt

[0074] raw:value

[0075] }

[0076] head＝nxt

[0077] (2.5)更新cache的entry_map：为entry_map添加一个新的条目，该条目包括键及其对应的键值，键为DNS请求报文携带的域名的哈希值，键值为<首个存储块索引，请求内容的大小，响应内容的大小，响应内容的过期时间，上次请求当前域名的时间>。

[0078] 本实施例中，该基于eBPF在内核层面的DNS缓存方法的具体工作过程包括如下内容：

[0079] 首先，进入初始化加载阶段，先初始化加载一些必要的程序或者配置文件等等。

[0080] S1、加载eBPF map:加载eBPF map,并调用bpf_obj_pin将map的文件描述符绑定到文件系统中的指定路径。例如将data_store这个eBPF map绑定到文件系统中的指定路径,即/sys/fs/bpf/entry_map路径,其他的程序就可以通过访问/sys/fs/bpf/entry_map路径读写这个eBPF map。

[0081] 其中,需要加载的eBPF map包括:①data_store,其类型为BPF_MAP_TYPE_ARRAY,值为存储块,每个块包括响应数据的一部分和指向下一块的index;②entry_map,其类型为BPF_MAP_TYPE_HASH,键为域名请求的哈希值,值为对应的元数据,包括响应在data_store中的起始索引、请求大小、响应大小、过期时间和上一次访问时间;③ring_buffer,其类型为BPF_MAP_TYPE_RINGBUF,用于在TC程序中向控制面发送DNS响应,即将eBPF程序中采集到的数据发送到用户态。

[0082] S2、加载数据面:包括加载TC程序、加载XDP程序、设置eBPF程序尾调用。

[0083] S21、加载TC程序:在网卡上创建clsact队列(qdisc),之后将tc_tx_filter程序以direct action模式加载到网卡的出口(egress)方向。

[0084] S22、加载XDP程序:将xdp_rx_filter程序加载到网卡上。

[0085] S23、设置eBPF程序尾调用:更新TC和XDP尾调用所使用到的eBPF program map。将tc_tx_update_cache和xdp_rx_response程序添加到该map中。

[0086] S3、加载控制面:启动控制面程序。

[0087] 然后,进入正式的运行阶段,实现本发明所述的在内核层面的DNS缓存方法。

[0088] S4、数据面:DNS服务器采用XDP程序。

[0089] S41、捕获并解析DNS客户端的DNS请求:xdp_rx_filter程序监听网卡上接收到的所有数据包,对数据包进行过滤得到DNS请求报文,并对DNS请求报文进行解析,计算请求内容的哈希值。

[0090] S42、查找缓存:将步骤S41计算的请求哈希值作为键,在entry_map中查找对应的元信息。若找到且未过期,则cache命中,更新该键的上次请求时间为现在(now),之后触发尾调用,执行xdp_rx_response程序,然后执行步骤S43;否则,说明cache未命中,交由DNS服务器处理,直接跳至步骤S5。

[0091] 应当理解的是,调用eBPF的bpf_ktime_get_ns函数获取当前时间,如果元信息中的过期时间大于当前时间就说明过期了。

[0092] S43、快速返回DNS响应:xdp_rx_response程序根据entry_map中的元信息的内容,从data_store中读取DNS响应,构造DNS响应报文,并返回XDP_TX,将DNS响应报文直接从网卡上反向发出。

[0093] 应当理解的是,根据元信息以及响应内容修改DNS请求报文,修改后的DNS请求报文即为构造的DNS响应报文。

[0094] S5、数据面:DNS客户端采用TC程序。

[0095] S51、捕获并解析DNS服务器的DNS响应:tc_tx_filter程序监听协议栈发出的报文,过滤并解析DNS服务器发出的DNS响应。

[0096] S52、将解析内容发送至控制面:tc_tx_filter通过尾调用执行tc_tx_update_cache程序,tc_tx_update_cache程序会将解析内容发送至ring_buffer。

[0097] S6、控制面:

[0098] S61、控制面获取DNS响应:控制面程序监听ring_buffer,在有新的DNS响应时会读取并进行后续步骤的处理。

[0099] S62、查找域名是否已缓存:以请求内容哈希为键,在entry_map中查找是否存在对应的元信息,若元信息已存在且过期,则删除entry_map和data_store中对应的内容;否则,直接执行步骤S63。

[0100] S63、检测data_store的剩余空间是否足够存储该DNS响应,若足够存储该DNS响应,则直接执行步骤S64;否则,则使用近似LRU方法驱逐旧的缓存条目,直到data_store空间足够。

[0101] S64、将DNS响应分为固定大小的多个存储块,将多个存储块存储至data_store中,具体可参见图2。

[0102] S65、更新entry_map:添加一个新的条目,该条目包括键及其对应的键值,键为请求内容哈希,键值为<首个存储块索引,请求内容的大小,响应内容的大小,响应内容的过期时间,上次请求当前域名的时间>。

[0103] 下面根据实施例详细描述本发明的基于eBPF在内核层面的DNS缓存方法,本发明的目的和效果将变得更加明显。

[0104] 初始状态下cache为空。

[0105] 客户端向DNS服务器发送一个DNS请求,其中请求解析的域名为example.com。

[0106] 该DNS请求被xdp_rx_filter捕获,过滤并解析DNS请求的内容并计算哈希值。

[0107] 以计算的哈希值为key,在entry_map中查找对应的元信息,由于目前没有任何缓存条目,所以cache miss,DNS请求被交由上层DNS服务器处理。应当理解的是,要访问的数据在cache中有缓存,称为“命中(hit)”,反之则称为“miss(缺失)”,表示未命中。

[0108] 上层DNS服务器返回响应。

[0109] tc_tx_filter捕获到该DNS响应,解析DNS响应内容并计算请求内容哈希值,通过尾调用执行tc_tx_update_cache程序。

[0110] tc_tx_update_cache程序将解析的DNS响应内容发送至ring_buffer。

[0111] 控制面程序监听到ring_buffer有新的响应。

[0112] 控制面程序以请求哈希值作为key,查找entry_map,发现没有对应的元信息。

[0113] 检查data_store的剩余空间,足够存储该DNS响应。

[0114] 将DNS响应分为固定大小的小块,存储至data_store中。

[0115] 更新entry_map,添加一个新的条目,key为请求内容哈希,value为<首个存储块索引,请求大小,响应大小,过期时间,上次请求时间>。

[0116] 客户端再次发送DNS请求,请求解析的域名为example.com。

[0117] 该DNS请求被xdp_rx_filter捕获,解析请求内容并计算哈希值。

[0118] 以计算得到的哈希值为key,在entry_map中查找对应的元信息,cache命中。

[0119] 根据元信息,从data_store中读取对应的响应内容,构造DNS响应报文,返回XDP_TX,从网卡反向发出,无需经过内核协议栈处理。

[0120] 过了一段时间之后,客户端再次发送DNS请求,请求解析的域名为example.com。

[0121] 该DNS请求被xdp_rx_filter捕获,解析请求内容并计算哈希值。

[0122] 以计算得到的哈希值为key,在entry_map中查找对应的元信息,发现当前已超过

元信息中记录的过期时间,因此cache miss,DNS请求被交由上层DNS服务器处理。

[0123] 上层DNS服务器返回响应。

[0124] tc_tx_filter捕获到该响应,解析响应内容并计算请求内容哈希值,通过尾调用执行tc_tx_update_cache程序。

[0125] tc_tx_update_cache程序将解析的响应内容发送至ring_buffer。

[0126] 控制面程序监听到ring_buffer有新的响应。

[0127] 控制面程序以请求哈希值作为key,查找entry_map,发现当前已超过元信息中记录的过期时间,将过期的响应内容从data_store中删除。

[0128] 检查data_store的剩余空间,足够存储该DNS响应。

[0129] 将DNS响应分为固定大小的小块,存储至data_store中。

[0130] 更新entry_map,添加一个新的条目,key为请求内容哈希,value为<首个存储块索引,请求大小,响应大小,过期时间,上次请求时间>。

[0131] cache空间已满,客户端发送DNS请求,请求解析的域名为abc.com。

[0132] 该请求被xdp_rx_filter捕获,解析请求内容并计算哈希值。

[0133] 以上一步计算的哈希值为key,在entry_map中查找对应的元信息,cache miss,DNS请求被交由上层DNS服务器处理。

[0134] 上层DNS服务器返回响应。

[0135] tc_tx_filter捕获到该响应,解析响应内容并计算请求内容哈希值,通过尾调用执行tc_tx_update_cache程序。

[0136] tc_tx_update_cache程序将解析的响应内容发送至ring_buffer。

[0137] 控制面程序监听到ring_buffer有新的响应。

[0138] 控制面程序以请求哈希值作为key,查找entry_map,发现没有对应的元信息。

[0139] 检查data_store的剩余空间,发现不够存储该DNS响应,执行近似LRU驱逐算法,将驱逐的内容从data_store和entry_map中删除,直到空间足够。

[0140] 将DNS响应分为固定大小的小块,存储至data_store中。

[0141] 更新entry_map,添加一个新的条目,key为请求内容哈希,value为<首个存储块索引,请求大小,响应大小,过期时间,上次请求时间>。

[0142] 本发明使用XDP程序在网卡层解析并响应DNS请求,无需经过内核协议栈,实现了内核协议栈之前的DNS缓存,在网卡驱动层直接响应DNS请求,避免内核协议栈的瓶颈;本发明使用TC程序捕获并解析DNS响应内容,通过BPF_MAP_TYPE_RINGBUF类型的eBPF map发送给控制面(用户态)处理;本发明通过控制面(用户态)监听BPF_MAP_TYPE_RINGBUF类型的eBPF map,异步地将响应存储至cache中,避免内核态eBPF程序的诸多限制,可支持多种cache驱逐算法;本发明使用eBPF map作为缓存存储,并在eBPF map中实现离散分块存储,极大提升内存利用率;本发明显著降低了客户端的请求延迟,提升了DNS解析速度;本发明显著降低了服务端的CPU消耗,提高服务端能承受的最大QPS,提升服务健壮性和可靠性;本发明无需修改DNS服务器和内核,对上层完全透明;本发明可动态加载和卸载。

[0143] 以上实施例仅用以说明本发明的技术方案,而非对其限制;尽管参照前述实施例对本发明进行了详细的说明,本领域的普通技术人员应当理解:其依然可以对前述各实施例所记载的技术方案进行修改,或者对其中部分技术特征进行等同替换;而这些修改或者

替换,并不使相应技术方案的本质脱离本发明各实施例技术方案的精神和范围。

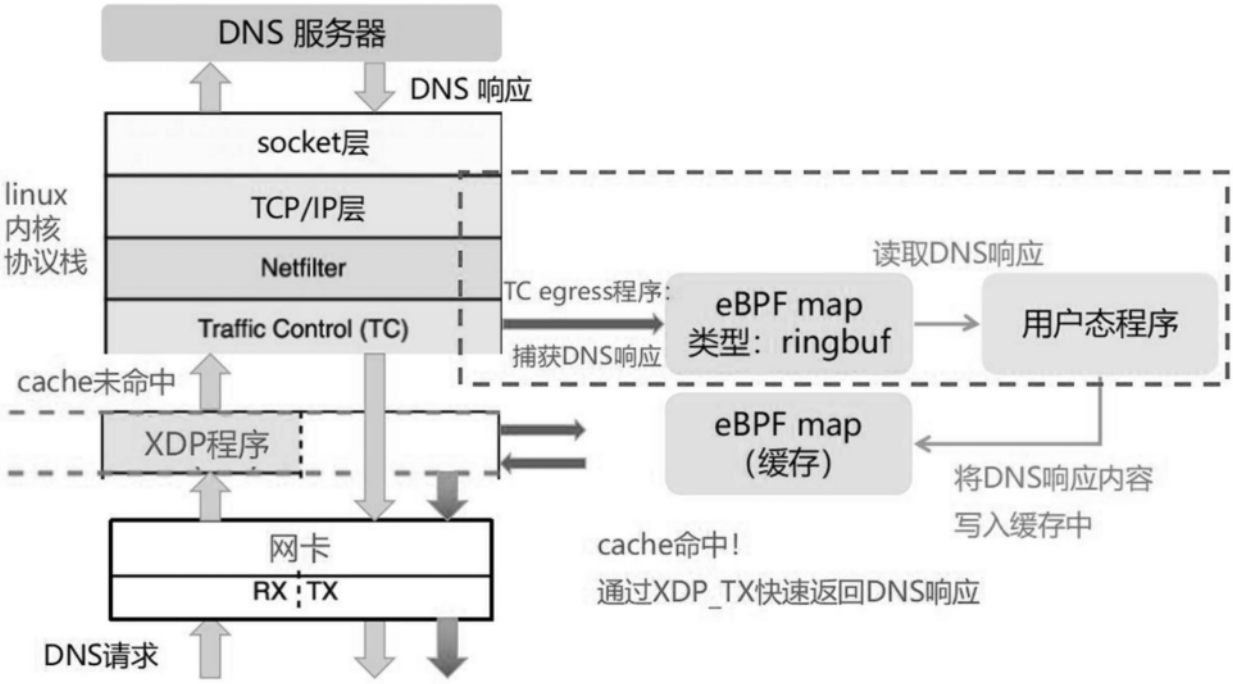

DNS 服务器
DNS 响应
socket层
linux
内核
协议栈
TCP/IP层
Netfilter
Traffic Control (TC)
读取DNS响应
TC egress程序:
捕获DNS响应
eBPF map
类型：ringbuf
用户态程序
cache未命中
XDP程序
eBPF map
(缓存)
将DNS响应内容
写入缓存中
网卡
RX TX
cache命中!
通过XDP_TX快速返回DNS响应
DNS请求

图1

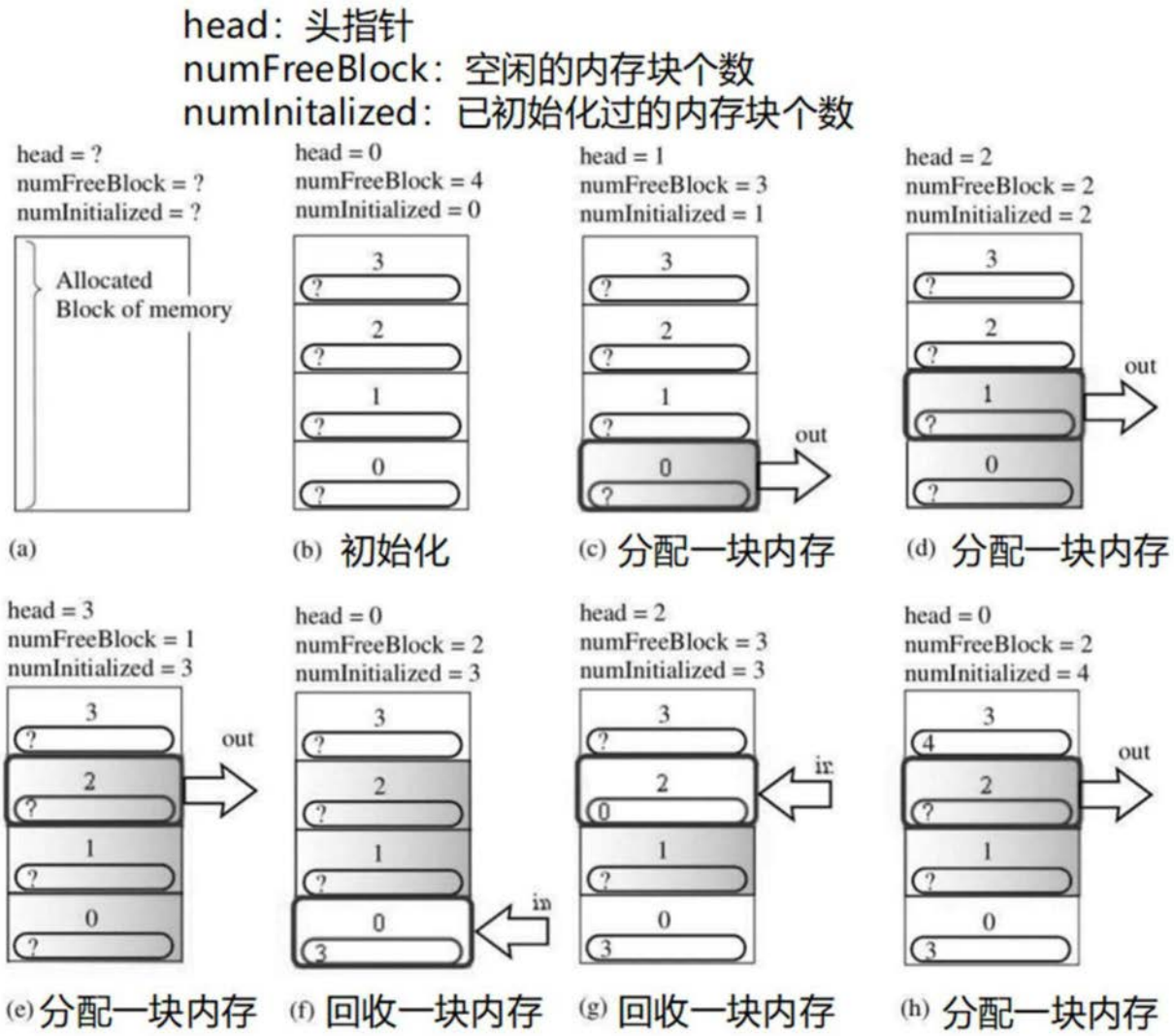

head：头指针
numFreeBlock：空闲的内存块个数
numInitalized：已初始化过的内存块个数
head = ?
numFreeBlock = ?
numInitialized = ?
Allocated
Block of memory
(a)
head = 0
numFreeBlock = 4
numInitialized = 0
(b) 初始化
head = 1
numFreeBlock = 3
numInitialized = 1
out
(c) 分配一块内存
head = 2
numFreeBlock = 2
numInitialized = 2
out
(d) 分配一块内存
head = 3
numFreeBlock = 1
numInitialized = 3
out
(e) 分配一块内存
head = 0
numFreeBlock = 2
numInitialized = 3
in
(f) 回收一块内存
head = 2
numFreeBlock = 3
numInitialized = 3
in
(g) 回收一块内存
head = 0
numFreeBlock = 2
numInitialized = 4
out
(h) 分配一块内存

图2